Retail Sales ETL Pipeline and Data Warehouse

Final Project Report

Mohamed Moin Peer

MSc Data Analytics, Queen's University Belfast

Portfolio project | 2026

Abstract

This report describes the design, implementation and evaluation of an end-to-end extract, transform and load (ETL) pipeline that converts raw retail sales exports into a star-schema data warehouse. The pipeline is written in Python and SQL and runs on SQLite by default or PostgreSQL through a single configuration change. A synthetic dataset of 110,076 raw transactions for a fictional UK retail chain, deliberately seeded with realistic data-quality defects, was used to evaluate it. The pipeline removed 1,086 duplicate transactions, quarantined 2,457 invalid rows (2.23%) with recorded reasons, and loaded 106,533 clean transactions in a single atomic transaction. Post-load reconciliation confirmed that row counts and monetary totals matched exactly. The complete run took approximately 5 seconds on a laptop, and 19 automated tests verify the validation rules, calculations and end-to-end behaviour, including idempotent re-runs.

1. Introduction

Retail organisations rely on sales data from point-of-sale (till) systems to make decisions about stock, staffing and promotions. These systems typically export transactions as CSV files. In practice, such exports contain errors: transactions scanned twice, missing store identifiers, impossible dates, quantities typed as text and mistyped prices. When these files are loaded directly into reports, revenue is overstated, comparisons between stores are distorted and confidence in the data declines.

The aim of this project was to build a pipeline that guarantees that every record reaching the warehouse has been validated, and that every rejected record is retained with a reason so that the problem can be fixed at source. The project focuses on data-engineering practice: modular design, data-quality rules, dimensional modelling, transactional loading, observability and automated testing.

1.1 Objectives

Objective	Success criterion
Trustworthy data	No invalid row is loaded; every rejected row has at least one recorded reason
Reliability	Loads are all-or-nothing and re-running produces identical results
Observability	Each run is logged with timings and recorded in an audit table
Portability	Runs on SQLite with no setup; PostgreSQL via one environment variable
Verifiability	Automated tests cover rules, calculations and the full pipeline

2. Data

No real company data was used. A generator module (generate_data.py) creates one year (2025) of synthetic sales for a fictional chain of 12 stores across the UK and Northern Ireland, selling 60 products in six categories. Monthly volumes follow a seasonal pattern (peaks in November and December, deeper discounts in the January and July sales), and store volumes are weighted by a size factor. A fixed random seed makes the dataset reproducible.

To test the pipeline realistically, approximately 3% of rows are corrupted on purpose. Table 1 lists the injected defects. The output is twelve monthly files (sales_2025_01.csv to sales_2025_12.csv) plus product and store reference files.

Injected defect	Real-world cause	Approx. share
Duplicate transaction rows	Item or basket scanned twice	1.0%
Missing store ID	Misconfigured till	0.6%
Unknown product code (P9999)	Product not set up in master data	0.4%
Negative quantity	Return keyed as a sale	0.5%
Text quantity ('two')	Manual override	0.3%
Impossible date (31/02/2025)	Wrong format or keying error	0.3%
Price outlier (£9,999.99)	Mistyped price	0.2%
Upper-case names with trailing spaces	Inconsistent system exports	5.0%

Table 1. Data-quality defects injected into the synthetic data.

3. System design

The pipeline consists of five stages, each implemented as a separate Python module and coordinated by pipeline.py (Figure 1). Separating the stages keeps each one small, testable and replaceable.

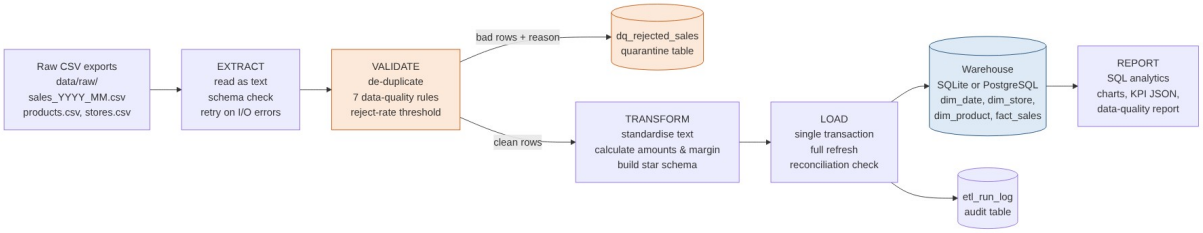


Figure 1. Pipeline architecture.

3.1 Data model

The warehouse uses a star schema (Figure 2), the standard design for analytical workloads. The central table, fact_sales, holds one row per transaction with its measures (quantity and the gross, discount, net, cost and margin amounts). Three dimensions, dim_date, dim_store and dim_product, provide the context for analysis. Integer surrogate keys, primary and foreign keys, a CHECK constraint on quantity and indexes on the join columns enforce integrity and support fast queries. The DDL in sql/schema.sql runs unchanged on both SQLite and PostgreSQL. Two further tables support operations: dq_rejected_sales (quarantined rows and reasons) and etl_run_log (one audit row per run).

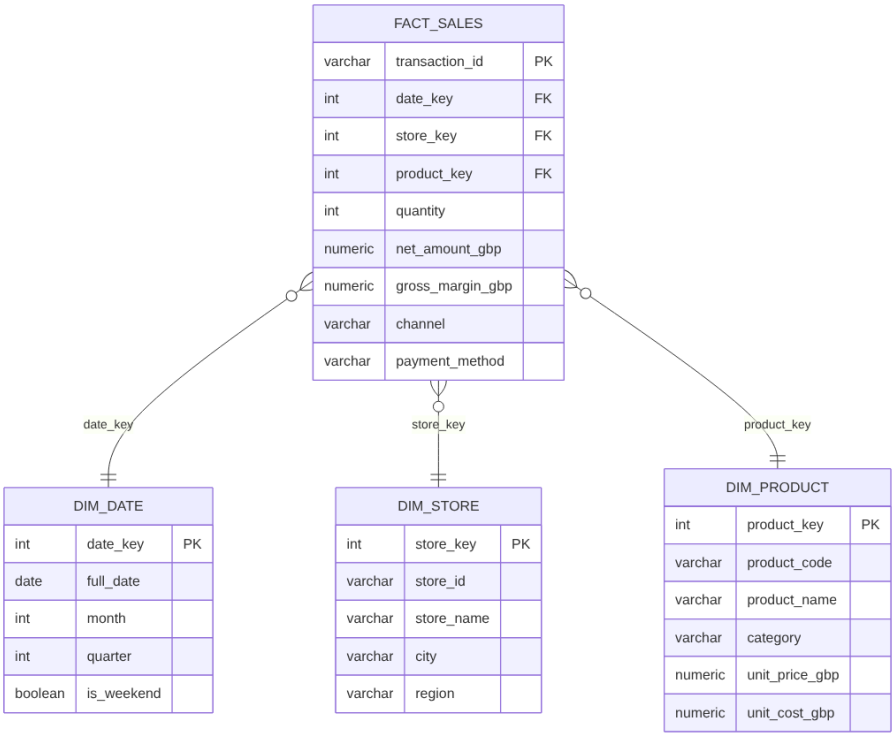


Figure 2. Star schema of the warehouse.

4. Implementation

4.1 Extract

All monthly files are discovered and read with every column typed as text. This design choice prevents a single malformed value from aborting the read; type enforcement is deferred to validation, where failures become rejected rows rather than crashes. Each file is checked for the ten required columns, and each row records its source file for lineage. File reads are wrapped in a retry decorator with exponential backoff (1, 2 and 4 seconds), which protects against transient errors when files are held on network storage.

4.2 Validate

Duplicate transaction IDs are removed first, keeping the earliest occurrence. Seven rules are then evaluated for every row (Table 2). A row may fail several rules; all reasons are recorded, separated by semicolons. Rejected rows are written to the quarantine table and to rejected_rows.csv rather than being discarded. A batch-level safeguard stops the run before loading if more than 10% of rows fail, because such a high rate usually indicates a broken export, for example a changed file layout.

Rule	A row fails when
missing_store_id	store_id is blank
unknown_store_id	store_id is not in the store reference file
unknown_product_code	product_code is not in the product reference file
invalid_quantity	quantity is not numeric, or is zero or negative

invalid_datetime	timestamp does not parse as YYYY-MM-DD HH:MM:SS (e.g. 31 February)
price_out_of_range	price is missing, not positive, or above £500
invalid_discount	discount is below 0% or above 90%

Table 2. Validation rules.

4.3 Transform

Text fields are standardised (trimmed store names; lower-case payment method and channel). Monetary measures are calculated once, centrally, and rounded to the penny: gross = quantity × unit price; discount = gross × discount rate; net = gross − discount; cost = quantity × unit cost; gross margin = net − cost. Calculating these in the pipeline gives every report the same definition of revenue and margin. The transform also generates a continuous date dimension covering every calendar day in the data.

4.4 Load

The load executes inside a single database transaction: the schema is rebuilt and all tables are inserted, dimensions before facts to respect foreign keys, and the transaction either commits completely or not at all. The full-refresh design makes the pipeline idempotent: running it twice on the same input produces an identical warehouse. After loading, a reconciliation query compares the row count and total net revenue in the database with the transformed data; any difference fails the run.

4.5 Reporting and observability

Six SQL queries in sql/analytics produce the reporting tables (monthly revenue, store performance, category mix, top products, hourly pattern and channel/payment split). Results are saved as CSV files and charts, alongside a KPI summary in JSON and a data-quality report in Markdown. Logs are written to the console and a rotating log file, including the duration of every stage and the count flagged by every rule, and each run (successful or failed) is recorded in etl_run_log. Configuration (database URL, thresholds, retry settings) is read from environment variables or a .env file, so no settings or credentials are hard-coded.

5. Testing

The test suite, written with pytest, contains 19 tests and runs in about four seconds (Table 3).

Area	Tests	What is verified
Validation	12	Each rule rejects with the correct reason; duplicates removed once; multiple reasons recorded; threshold stops a bad batch
Transform	3	Monetary calculations to the penny; text standardisation; complete date dimension
Utilities	2	Retry succeeds after transient failures and gives up after the limit

End to end	2	Full run into a temporary database: no orphan facts, SUCCESS in the audit log, outputs created, identical results on re-run

Table 3. Automated test coverage.

6. Results

6.1 Pipeline performance and data quality

Metric	Value
Raw rows extracted (12 files)	110,076
Duplicate transactions removed	1,086
Rows rejected and quarantined	2,457 (2.23%)
Rows loaded to fact_sales	106,533
Reconciliation	Passed: rows and £ totals match
End-to-end runtime	~5 seconds
Automated tests	19 passed

The reject rate of 2.23% is well below the 10% threshold, so the batch was accepted. Figure 3 shows the defects detected. The counts align with the injected defect rates in Table 1, which indicates that every class of defect was detected. Invalid quantities were the largest category of rejected rows.

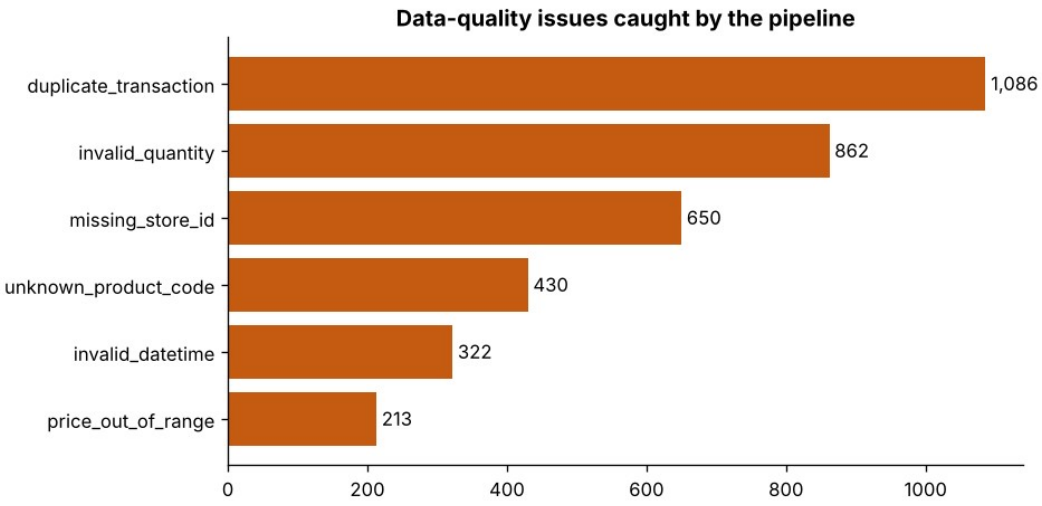


Figure 3. Data-quality issues caught by the pipeline.

6.2 Business reporting

Total net revenue was £6.94 million with a gross margin of 34.8% across 106,533 transactions (average £65.11). December was the strongest month (£900k, 13.0% of the year), and margin fell to about 27% in

January and July as a result of sale discounts (Figure 4). London Oxford St was the highest-revenue store, followed by Belfast City Centre, and Electronics was the largest category, while Beauty had the highest margin percentage (39.0%).

Because the data is synthetic, these patterns reflect the assumptions built into the generator. Their value is in showing that the warehouse and SQL layer correctly surface seasonality, discount effects and store and category differences; they are not claims about real market behaviour.

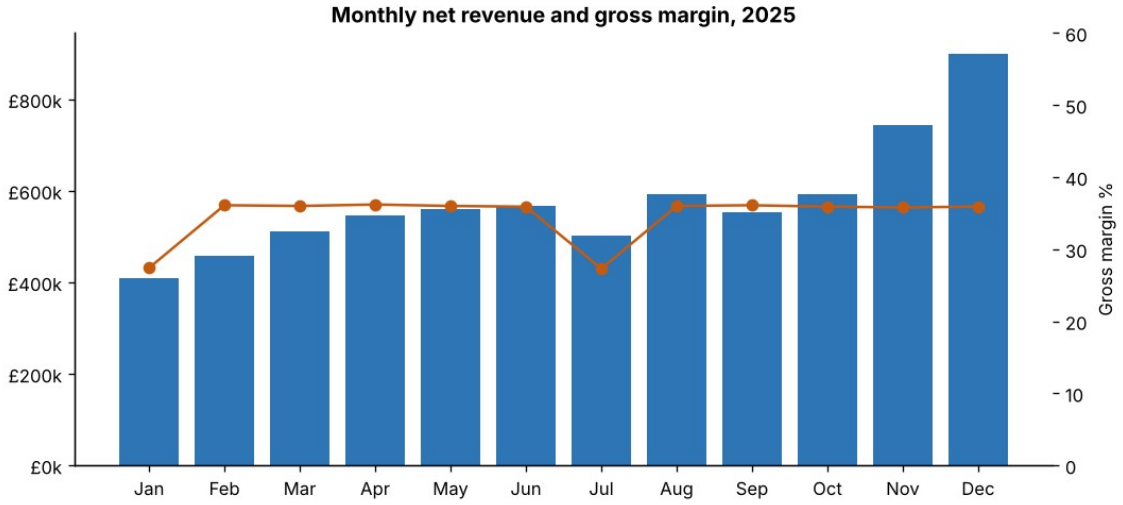


Figure 4. Monthly net revenue and gross margin.

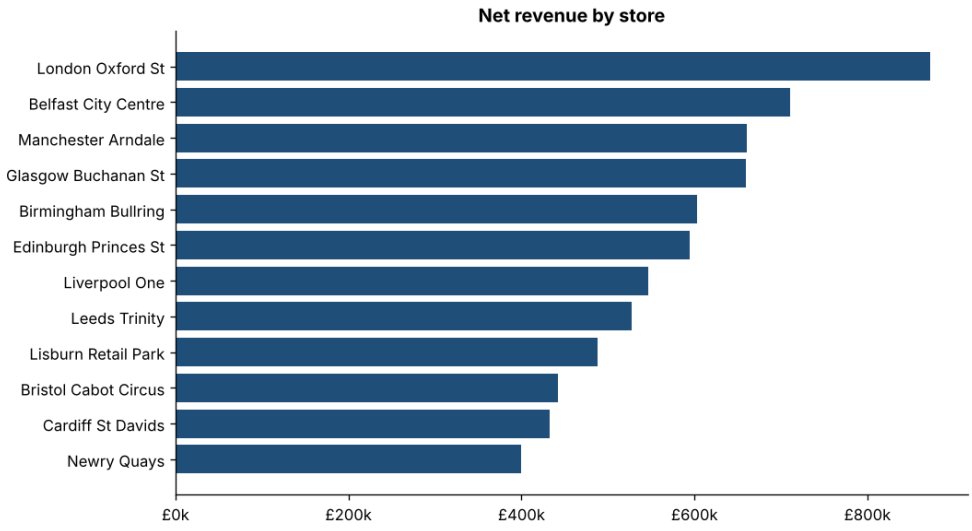


Figure 5. Net revenue by store.

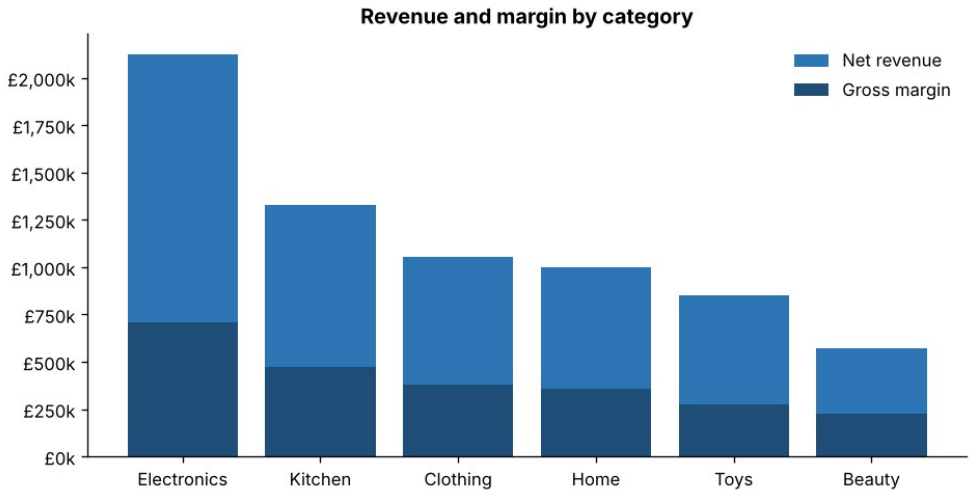


Figure 6. Revenue and margin by category.

7. Discussion

The most valuable part of the pipeline is the validation layer. Deciding what counts as invalid, recording why, and retaining rejected rows for correction is what turns a file loader into a trustworthy data source. Reading all inputs as text and enforcing types during validation proved to be an effective way of handling dirty data without crashes.

Making the load transactional and idempotent removes a whole category of operational failure: a failed run cannot leave a half-loaded warehouse, and an operator can safely re-run the pipeline after fixing a problem. The reconciliation check provides a final, independent confirmation that what was loaded matches what was intended. The test suite allowed the code to be refactored with confidence.

7.1 Limitations

- The data is synthetic, so the business findings are illustrative rather than real insights.
- The full-refresh strategy suits a year of data but would not scale efficiently to very large volumes.
- Dimension changes (for example a store moving region) overwrite history rather than preserving it.
- There is no scheduler or alerting; the pipeline runs on demand.

8. Future work

- Incremental loading of new monthly files only, tracked with a file-registry table.
- Type-2 slowly changing dimensions for store and product history.
- Orchestration with Apache Airflow or Prefect, with a daily schedule and failure alerts.
- Declarative data contracts using Great Expectations or pandera.
- A Power BI dashboard connected to the warehouse for business users.

9. Conclusion

The project delivered a modular, tested ETL pipeline that meets all five objectives. It detected every class of injected defect, quarantined 2,457 invalid rows with reasons, removed 1,086 duplicates, and loaded 106,533 clean transactions into a reconciled star-schema warehouse in about 5 seconds. The design choices, text-first extraction, rule-based quarantine, transactional idempotent loading, reconciliation and automated testing, reflect practices used in production data engineering and provide a solid base for the extensions described above.

Appendix A. How to run

```
python -m venv .venv
.venv\Scripts\activate        (macOS/Linux: source .venv/bin/activate)
pip install -r requirements.txt
pip install -e .
python -m retail_etl all
pytest
```

Appendix B. Repository structure

Path	Contents
src/retail_etl/	Pipeline package: config, extract, validate, transform, load, analytics, pipeline, CLI
sql/schema.sql	Warehouse DDL (SQLite and PostgreSQL compatible)
sql/analytics/	Six reporting queries
tests/	19 pytest tests
outputs/	Charts, result tables, KPI summary, data-quality report
docs/	Architecture diagrams, presentation, this report